

★ Playwright Top 35 Interview Questions ★

1. What is Playwright, and how is it different from Selenium?

→ Playwright is an end-to-end testing framework by Microsoft that supports multiple browsers, auto-waiting, parallel execution and powerful APIs.

Unlike Selenium, it has auto-wait by default, supports multiple browsers with one API, and can test modern web apps (SPA, Shadow DOM, etc.) easily.

2. Which browsers and languages does Playwright support?

→ Playwright supports Chromium, Firefox and WebKit (Safari).

It supports JavaScript, TypeScript, Python, Java, .NET (C#) languages.

3. What is auto-waiting in Playwright, and how does it reduce flaky tests?

→ Auto-waiting means Playwright automatically waits for elements to be ready (visible, stable, enabled) before performing actions. This reduces flaky tests caused by timing issues and manual waits.

4. What is the difference between a Page and a BrowserContext in Playwright?

- Page represents a single tab or window in a browser.
- BrowserContext provides isolated environment for multiple pages, cookies, storage, permissions etc.

Each test should run in a new BrowserContext for isolation.

5. How do you launch a browser in headless vs headed mode?

-
- Headless mode: Browser runs in background (no UI).
 - Headed mode: Browser opens with UI.
 - Example:

```
const browser = await playwright.chromium.launch({ headless: true }); // headless
const browser = await playwright.chromium.launch({ headless: false }); // headed
```


★ Playwright Top 35 Interview Questions ★

6. What are locators in Playwright, and why are they preferred over raw selectors?

→ **Locators** are a way to find elements on the page.

They are preferred because they are **resilient**, **readable** and **automatically wait** for the element to be ready before performing actions.

They reduce flaky tests caused by dynamic UI changes.

Example: `const button = page.getByRole('button', { name: 'Submit' });`

7. How do you handle explicit waits when auto-wait isn't enough?

→ We can use explicit wait methods like `waitForSelector()`, `waitForTimeout()`, `waitForFunction()`, `waitForLoadState()`, `waitForResponse()`, etc.

Example:

`await page.waitForSelector('#loader', { state: 'hidden', timeout: 10000 });`

8. What is the difference between `click()` and `dispatchEvent()` in Playwright?

- `click()` simulates a real user click. It triggers all the default browser behaviors, waits for element to be ready, and works with auto-wait.
- `dispatchEvent()` triggers the event directly on the element without any actionability checks or default browser behaviors.

9. How do you take screenshots and record videos of test runs in Playwright?

→ **Screenshots:**

- Take screenshot of a page or an element.

`await page.screenshot({ path: 'screenshot.png', fullPage: true });`

→ **Videos:** Enable video recording in the context.

- Configure while creating context.

`const context = await browser.newContext({ recordVideo: { dir: 'videos/' } });`

10. What is the default timeout in Playwright, and how do you configure it?

→ Default timeout is **30 seconds** for most actions.

We can configure timeouts at different levels.

- **Global timeout for tests:** `test.setTimeout(60000);` // for a test
- **Action timeout:** `page.setDefaultTimeout(15000);` // for actions
- **Navigation timeout:** `page.setDefaultNavigationTimeout(30000);` // for navigation

★ Playwright Top 35 Interview Questions ★

11. How do you run tests in parallel using Playwright?

→ Playwright runs tests in **parallel by default**.

- Parallelism is configured in **playwright.config** file.
- Each test file runs in a separate worker process.

Example :

```
export default defineConfig({
  workers: 4,      // run tests in 4 parallel workers
});
```

12. What is the Playwright Test Runner, and how is it different from just using the Playwright library?

→ Playwright Test Runner is a **built-in test framework** on top of the Playwright library for writing, running and reporting tests.

- Provides **test structure, fixtures, parallel execution, reporters, tracing, retries**, etc.
- Using just the library is lower-level - you have to manage all setup, teardown, reporting yourself.

13. How do you handle multiple tabs/windows in a single test?

→ Use **context.waitForEvent('page')** to capture the new tab/window.

Example :

```
const [newPage] = await context.waitForEvent('page');
await newPage.waitForLoadState('load');
```

14. How do you interact with elements inside an iframe using Playwright?

→ Use **frameLocator()** to directly locate elements inside an iframe.

Example :

```
const frame = page.frameLocator('#myFrame');
await frame.locator('#submit').click();
```

15. **Scenario:** A button click triggers an API call that updates the UI after 3 seconds. How would you wait for and assert on that update reliably?

→ Use **waitForResponse()** to wait for the API call and then assert UI.

- Don't use hard sleeps.
- Wait for the API to complete and then verify the updated UI.

Example :

```
await Promise.all([
  page.waitForResponse('**/api/order-status'),
  page.locator('#updateBtn').click()
]);
await expect(page.locator('.status')).toContainText('Updated');
```


16. What are fixtures in Playwright, and how do you create a custom fixture?

→ Fixtures are **reusable setup/teardown** logic that can be shared across tests.

- **Built-in fixtures**: page, context, browser, etc.
- **Custom fixtures** are created using the **test.extend()** method.

Example:

```
const test = base.extend({
  loginPage: async ({ page }, use) => {
    const lp = new LoginPage(page);
    await lp.goto();
    await use(lp);
  }
});
```

17. How do you handle file uploads and downloads in Playwright?

→ **File Upload**: Use **setInputFiles()** on the file input element.

Example: `await page.locator('#fileInput').setInputFiles('path/to/file.pdf');`

→ **File Download**: Use **waitForEvent('download')** and save the file.

Example:

```
const [ download ] = await Promise.all({
  page.waitForEvent('download'),
  page.click('#downloadBtn')
});
await download.saveAs('path/to/save/file.pdf');
```

18. Scenario: One of your tests fails intermittently in CI but passes locally. How would you debug and fix this flakiness?

- **Steps**:
- 1) Reproduce in CI locally
 - 2) Collect traces / videos / screenshots
 - 3) Check for race conditions, timing issues, hard waits
 - 4) Use auto-wait, proper locators, retries for network, stable test data.
 - 5) Fix root cause and re-run in CI to confirm stability.

19. What is the Page Object Model (POM), and how do you implement it in Playwright?

→ POM is a **design pattern** where page-specific actions and locators are encapsulated in separate classes.

- Improves **maintainability**, **reusability** and **readability**.

Example: Create a class for each page and expose methods for actions.

20. How do you intercept and mock network requests/responses in Playwright?

→ Use **page.route()** to intercept and modify requests or mock responses.

Example:

```
await page.route('**/api/users', route => {
  route.fulfill({ status: 200, body: JSON.stringify({ users: [] }) });
});
```


21. How do you handle authentication once and reuse the session across multiple tests?

→ We can **save the authenticated storage state** (cookies, localStorage, etc.) and reuse it in other tests.

- Login once, save storage state to a file.
- Use that file while creating new context.

Example :

```
// Login test
await context.storageState({ path: 'state.json' });

// Reuse in other tests
const context = await browser.newContext({ storageState: 'state.json' });
```

22. What is the difference between soft assertions and hard assertions in Playwright?

→ **Hard assertions** (expect) will fail the test immediately if the condition is not met.

Soft assertions (expect.soft) will **record the failure** but **continue** executing the remaining steps in the test.

Example :

```
expect(locator).toHaveText('Hello'); // Hard assertion
expect.soft(locator).toBeVisible(); // Soft assertion
```

23. How do you run the same test across multiple browsers (Chromium, Firefox, WebKit)?

→ Use Playwright Test Runner's built-in **projects** feature in the config file.

Example :

```
projects: [
  { name: 'chromium', use: { browserName: 'chromium' } },
  { name: 'firefox', use: { browserName: 'firefox' } },
  { name: 'webkit', use: { browserName: 'webkit' } },
],
```

24. Scenario: You need to test a dynamic table where rows get added/removed based on filters. How would you validate row count and content reliably?

→ **Steps** to validate dynamic table reliably :

- 1) Apply filter and wait for table to refresh (e.g., wait for network or spinner to disappear).
- 2) Locate rows using locator ('table tbody tr').
- 3) Assert row count using toHaveCount().
- 4) Loop through rows and validate content.

25. How do you integrate Playwright with a CI/CD pipeline like Jenkins or GitHub Actions?

→ Install dependencies, run tests in headless mode and publish reports/artifacts.

- Configure Playwright in the pipeline (install Node, install deps, run tests).
- Generate reports (HTML / Allure / JUnit) and publish as build artifacts.
- Store screenshots/video/traces on failure for debugging.

26. What is trace viewer in Playwright, and how do you use it to debug failures?

→ Trace viewer helps to **inspect** the test execution step-by-step.

- It shows **DOM snapshots**, **network**, **console logs**, **screenshots**, **videos**, and actions performed.
- Very useful to identify root cause of failures.

Example :

```
// Enable trace in playwright.config.ts
use: { trace: 'on-first-retry' } // capture trace on failure
// View trace
npx playwright show-trace trace.zip // opens trace viewer
```

27. Scenario: How would you design a hybrid framework combining Playwright (UI) with REST Assured/API tests for end-to-end validation?

→ **Approach** : Use API tests for data setup/validation and UI tests for user flows.

- Use **REST Assured** (or similar) to create/validate data.
- Use Playwright for UI interaction and verification.
- Share common data and utilities between API and UI layers.

Flow : **API (create data) → UI (validate) → API (verify result)**

28. How do you handle shadow DOM elements in Playwright?

→ Use **pierce selector (>>)** or locator API with **shadow** option.

Example :

```
// Using >> to access shadow DOM
const element = page.locator('my-element').locator('>> .inner-element');
await element.click();
```

29. What is the difference between `page.route()` and `page.waitForResponse()`?

<code>page.route()</code>	<code>page.waitForResponse()</code>
• Used to intercept and modify or mock network requests. • Can block, modify, continue, fulfill requests. • Works before the request is sent.	• Used to wait for a specific network response. • Helps to wait until the response is received. • Does not modify the request.

30. Scenario: How would you test a feature flag-driven UI where behavior changes based on backend config, without hardcoding waits?

→ **Approach** : Don't use hard waits. Instead -

- Trigger feature flag via API (on/off).
- Reload UI and wait for the expected element/state using auto-wait or explicit waits.
- Assert the UI based on the flag.

Example :

```
// Enable feature flag via API
await request.post('/api/feature-flag', { data: { flag: 'new-dashboard', enabled: true } });
// Reload and wait for UI change
await page.reload();
await expect(page.locator('#newDashboard')).toBeVisible(); // auto-wait handles it
```


31. How do you manage test data and environment configs across dev/staging/prod in a Playwright framework?

→ Use **environment files** and **config manager** to load environment specific data and settings.

- Keep **base URL**, **timeouts**, **creds**, **API endpoints** in env files.
- Load data dynamically based on the environment.

Example :

```
// config.ts
export const config = {
  baseUrl: process.env.BASE_URL,
  username: process.env.USERNAME,
  password: process.env.PASSWORD
};
```

// Different .env files for
// dev, staging, prod

32. How do you generate and customize HTML/Allure reports for Playwright test runs?

→ Playwright has built-in **HTML report**. For **Allure**, use **allure-playwright**.

- HTML report is generated with "npx playwright show-report".
- Allure needs integration and generates **rich, customizable reports**.

Example :

```
# Run tests and generate Allure results
npx playwright test --reporter=allure-playwright
# Generate and open Allure report
allure generate allure-results --clean && allure open
```

33. Scenario: A test needs to validate a Kafka-triggered UI update (e.g., order status changes after a message is consumed). How would you approach this in Playwright?

→ **Approach**: Don't rely on fixed waits. Wait for backend state first, then validate UI.

- Check message processed via API/DB (or test hook).
- Once the backend state is updated, validate UI using **auto-wait**.
- Use **polling** with **timeout** if needed.

Example :

```
await expect.poll(async () => {
  const res = await request.get('/api/orders/123/status');
  return res.json().status;
}, { timeout: 30000 }).toBe('Delivered');
await expect(page.locator("#orderStatus")).toHaveText('Delivered');
```

// Wait for backend
// update first

// Then verify UI

34. How do you handle browser context isolation for running tests with different user roles in parallel?

→ Use **separate BrowserContext** for each user role.

- Each context has its own cookies, localStorage, session, etc.
- Run tests in parallel using test fixtures or workers.

Example :

```
const adminContext = await browser.newContext();
const userContext = await browser.newContext();
```

// Different contexts
// for different roles

35. What are some best practices you'd enforce in a large-scale Playwright framework to keep it maintainable and scalable?

→ **Best Practices** :

- Follow **Page Object Model (POM)**.
- Use **fixtures** for setup/teardown and reusability.
- Keep tests **independent** and **idempotent**.
- Use **meaningful names** for tests, locators, data.
- **Centralize** config, constants and test data.
- Use **proper waits** and avoid hard waits.
- Generate and review **reports/traces** regularly.
- Keep the framework **modular** and **DRY**.

COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



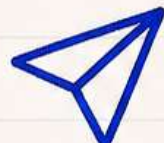
TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.



SAVE



FOLLOW



SHARE

— Thank you for your support! ❤️